Westminster University
School of Arts & Sciences | Salt Lake City

115 Weightlifting

Coach Programming, Athlete Execution, and Performance Tracking

A capstone report submitted in partial fulfillment
of the requirements for CMPT 390: Computer Science Capstone

Addy Cruz
Department of Computer Science & Data Science

Capstone instruction
Dr. Hu Helen

May 2026

Executive summary.

- **Product:** Role-aware web app for Olympic weightlifting teams (head coach, line coaches, athletes): programs, assignments, logs, PR history, roster analytics, Sinclair context.
- **Evidence:** Docker-first repo; automated frontend and backend tests; scripted UAT (`./115-weightlifting/bin/zw uat`); dated milestone and issue logs.
- **Run:** `docker compose up --build` from the repository root; ports and demo accounts documented in the README.
- **Scope:** Academic capstone quality and handoff; not a managed production service (monitoring, backups, and formal usability study out of scope for this release).

Figure 1: At-a-glance summary for reviewers and stakeholders.

Abstract

Small Olympic weightlifting teams often split programs, athlete logs, personal records, and review notes across spreadsheets, messages, and ad hoc files. 115 Weightlifting is a full-stack web application that centralizes role-aware workflows for head coaches, line coaches, and athletes: structured weekly programs, completion tracking, workout history, personal-record charts, roster aggregates, and Sinclair-aware analytics (International Weightlifting Federation 2026a, 2026b). The stack pairs a React and Vite frontend with a Django REST backend, JWT authentication, and a local SQLite workflow with PostgreSQL-ready configuration (Meta Open Source 2026; Vite Contributors 2026; Django Software Foundation 2026; Encode OSS Ltd. 2026; Jazzband 2026; SQLite Consortium 2026; PostgreSQL Global Development Group 2026). The repository records nineteen dated implementation milestones and thirteen dated issue resolutions, which this report uses as evidence that the delivered system matches iterative engineering work rather than a paper-only design.

Introduction

Olympic weightlifting depends on structured cycles, repeated exposure to the competition lifts, accessory work, and progressive loading. Coaches need to see what was assigned, what was completed, how personal records move, and whether training volume and intensity align with intent. Athletes need a simple daily view of prescribed work and a reliable way to log outcomes. When that information is fragmented, coordination across multiple coaches degrades and review meetings lose a single source of truth.

115 Weightlifting targets that fragmentation directly. It is not a generic fitness tracker; it encodes the relationship between head coach, line coaches, and athletes, and it bakes in weightlifting-specific concepts such as snatch, clean and jerk, total, competitive bodyweight class, and Sinclair scoring.

Industry or Organization Partner

This capstone is implemented as a reproducible, version-controlled repository with Docker-first local launch for faculty evaluation. There is no formal industry sponsor and no production deployment bound to an external organization. The "partner'' role is played by the coaching workflow itself: requirements were derived from real team operations, then scoped to what a single-term capstone can complete without compromising depth on program assignment, logging, and analytics.

Methods

Architecture and roles

The system uses a conventional client-server split. The frontend is a React application built with Vite and React Router, with Chart.js for time-series and comparison charts (Meta Open Source 2026; Vite Contributors 2026; Chart.js Contributors 2026). The backend is Django with Django REST Framework and SimpleJWT for bearer-token authentication (Django Software Foundation 2026; Encode OSS Ltd. 2026; Jazzband 2026). The default developer and demo workflow uses SQLite; environment configuration supports PostgreSQL for production-shaped deployments (SQLite Consortium 2026; PostgreSQL Global Development Group 2026).

Role-aware routing exposes three primary dashboards: `/head` for organization-level rollups, `/coach` for program authoring and assignment, and `/athlete` for consumption, completion, and personal analytics. A protected-route layer checks the authenticated user and redirects away from unauthorized areas. This design keeps policy close to the UI surface while the API enforces ownership and role rules server side.

Data modeling for flexible programs

Training programs store structured weekly content as JSON so the program builder can represent blocks, days, exercises, sets, repetitions, percentages, and notes without forcing every nested element into its own relational table. Programs remain assignable, versionable on the server, and joinable to athlete completion records and workout logs. The tradeoff, discussed later, is that some aggregate queries become application-level rather than a single SQL join.

Verification strategy

Verification combines automated tests, a scripted user-acceptance flow against a running stack, and manual multi-browser checks during demo preparation. Frontend and backend test suites cover authentication, interceptors, program templates, normalization utilities, and analytics endpoints. A bundled UAT script exercises registration, coach program create and update, athlete completion, workout logs, personal records, and the Sinclair calculation path end to end.

Results

Delivered feature set

The shipping application includes JWT login and registration, role-aware navigation, coach-side program create, edit, reassign, and inspect flows, athlete-side assigned-program views with completion persistence, workout log and personal record APIs, head-coach rollups, and weightlifting analytics including monthly

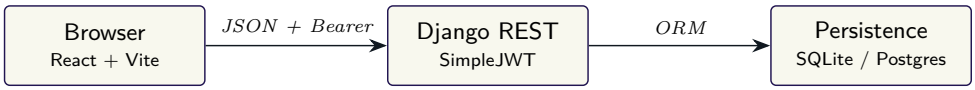

Figure 2: System context: the SPA authenticates to the API with JWT-backed requests; data stores are SQLite for local demo with a PostgreSQL-oriented configuration path for larger deployments.

Layer	Responsibility
React client	JWT-aware API client, dashboards, charts, program editor UX
Django REST	Users, programs, assignments, logs, PRs, analytics endpoints
Persistence	SQLite by default; PostgreSQL URL for large-tier compose
Operations	Docker Compose, seed scripts, `./115-weightlifting/bin/zw` automation

Table 1: High-level responsibilities by layer. The same decomposition is documented in the repository README and deployment guides.

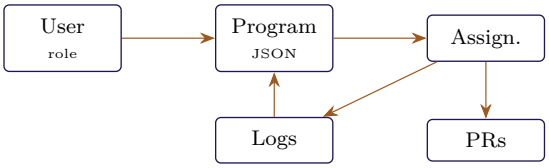

Figure 3: Simplified domain relationships (not a full ERD): users own programs; programs are assigned; assignments tie to workout logs and personal records; logs reference program context.

bests, rolling peak totals, roster aggregates, competitive weight class context, and Sinclair scoring (International Weightlifting Federation 2026b).

Demo database scale

Table 2 summarizes representative counts from the seeded demo database used for charts, roster views, and completion analytics. These are not production populations; they exist to prove the UI under realistic volume.

Implementation cadence

Nineteen dated milestones from late January through mid March 2026 move from environment setup to authentication, program APIs, athlete logging, analytics, demo tooling, UI facelift, and a documented API hardening backlog. Table 3 lists anchor dates; the repository feature log carries the full table with evidence notes.

Testing and scripted UAT

The frontend and backend ship dedicated test suites covering authentication, API contracts, spreadsheet-style program editing, charts, and analytics. Live counts are produced by `npm test` (frontend, Vitest) and `python manage.py test` (backend, Django test runner) and exceed two hundred automated checks at the time of writing; reviewers running the suites locally will see the current authoritative totals printed by the runners.

The scripted UAT flow that ships with the repository recorded a full pass of nineteen checks after the late-stage API stabilization work, covering registration and login, `/auth/me`, coach program create and update, athlete completion retrieval and save, workout logs, personal records, and the Sinclair endpoint. Separately, an auth stress report from April 24, 2026, ran eight login and logout cycles for a coach persona with zero failures, addressing an early project risk around token attachment and session stability.

Issue resolution pattern

Thirteen dated issues from January 31 through March 11, 2026, cluster into familiar full-stack themes: API base URL alignment, JWT header consistency, CORS configuration, serializer contract mismatches between coach forms and the backend, persistence for athlete logs, DisallowedHost behavior in automated smoke tests, local self-host dependency gaps, and presentation-time diagram rendering. Each entry pairs a symptom with a

Metric	Count
Users	67
Head coaches	2
Line coaches	3
Athletes	61
Training programs	90
Program completion records	62
Workout logs	6,387
Personal records	1,103

Table 2: Seeded demo counts at the time of the final report draft. Numbers fluctuate slightly if seed scripts change between commits.

Window	Theme
2026-01-29 to 2026-02-14	Skeleton, auth, models, first end-to-end login to programs
2026-02-15 to 2026-02-23	Program CRUD and assignment, athlete dashboards, MVP integration
2026-03-01 to 2026-03-11	Completion flow, Sinclair wiring, self-host scripts, UI pass, UAT

Table 3: Condensed milestone themes. The authoritative dated log lives alongside course deliverables in the shared capstone workspace.

Figure 4: Implementation cadence (approximate timeline): themes align with Table 3; exact dates are in the repository feature log.

Artifact	Outcome
`./115-weightlifting/bin/zw uat` (late capstone)	19 / 19 checks passed
Auth stress (2026-04-24)	8 / 8 cycles passed

Table 4: Representative scripted validation results. Re-run commands are documented in repository operator notes.

resolution date, which is how this report treats debugging as a first-class deliverable.

Discussion

This section states design intent and tradeoffs behind the delivered system: how program JSON, role-separated UIs, and analytics scope align with capstone constraints, and where that leaves room for future work.

Why JSON program documents

Storing program structure as JSON keeps the program builder flexible for coach-authored variations without migrations for every new prescription pattern. The API can still enforce ownership, assignment, and referential integrity for programs, athletes, and completion rows. The cost is heavier client responsibility for validation and more complex server-side reporting if future work needs deep SQL analytics across nested exercises. For the capstone window, the tradeoff favored iteration speed and demo fidelity.

Role separation versus convenience

Head, coach, and athlete routes intentionally duplicate almost no UI code paths. That duplication is a conscious readability choice for grading and maintenance: each dashboard optimizes for its persona rather than forcing a single configurable mega-view. The downside is that shared widgets must be refactored carefully when visual design changes, which showed up during the March UI facelift milestone.

Analytics scope

Charts and Sinclair tooling are designed as planning aids, not predictive sports-science models. Peak and rhythm views use conservative heuristics so coaches are not presented with false precision. That restraint is a product decision tied to trust: a coach who disagrees with a chart should still trust the underlying logged numbers.

Limitations

1. **Scope versus proposal.** Meet-day attempt management and richer competition workflows were deferred so that programming, logging, and analytics could reach production-quality depth inside one term.
2. **Deployment posture.** The professor-facing repository emphasizes Docker Compose and documented environment variables, but it does not ship managed hosting, backups, or monitoring. A real club deployment would need onboarding, account lifecycle, and data migration plans beyond the seeded demo.
3. **Usability evaluation depth.** Structured third-party usability sessions with recorded task times were not completed to publication standard by the draft date of this PDF. Manual walkthroughs during demo preparation did validate that three-browser, three-role presentations are feasible, and course review feedback should be folded into the next usability pass.

Conclusion and Future Work

115 Weightlifting delivers a coherent, role-aware system within academic scope: it connects coach programming to athlete execution and surfaces weightlifting-specific analytics, backed by automated tests, scripted UAT, and a transparent issue history.

Future work naturally splits into operations and product lanes. Operations would add hardened hosting, HTTPS, secrets management, backups, and observability. Product work would deepen periodization tooling, coach commentary on logs, notifications, optional meet-day modules, and first-class spreadsheet import and export for coaches who still live in Excel during transition.

Code and Data Availability

All application source, Docker definitions, validation drivers, and this Quarto report live in the version-controlled capstone repository at the repository root and under `115-weightlifting/`. Quick launch for reviewers is `docker compose up --build` with `.env` copied from `.env.example`; large-tier layout and ports are documented under `docs/`. Operator automation for local workflows is exposed through `./115-weightlifting/bin/zw`. Demo credentials and role walkthrough accounts are listed in the README. Course narrative artifacts, including the long-form dated feature and issue logs referenced throughout this report, are maintained in the CMPT-390 shared workspace deliverables tree alongside this repository.

Acknowledgements

I thank Dr. Hu Helen, my CMPT 390 instructor, for capstone structure, feedback, and guidance on iteration, testing, and honest scoping. That scaffolding is reflected in the milestone and issue logs cited here.

I extend my deepest gratitude to three faculty members whose mentorship shaped me throughout my time at Westminster University: Dr. Kathryn Lenth, Dr. Liang Jingsai, and Greg Gagne M.S. I am profoundly grateful for their teaching, mentorship, patience, and support across these years. They set the gold standard for the kind of educator I hope to keep learning from, and the rigor they demanded carried directly into the engineering decisions documented in this report. As I move from Westminster into the workforce, I leave not only prepared but fiercely competitive in an industry where masses are flooding interviews and job postings; that confidence is, in no small part, theirs.

I thank the Computer Science and Data Science faculty more broadly for the code review habits and systems thinking that made a full-stack term project tractable, and for years of feedback that turned coursework into engineering judgment.

Open-source libraries used include React (Meta Open Source 2026), Vite (Vite Contributors 2026), Django (Django Software Foundation 2026), Django REST Framework (Encode OSS Ltd. 2026), SimpleJWT (Jazzband 2026), and Chart.js (Chart.js Contributors 2026). Token handling follows common JWT practice (Jones, Bradley, and Sakimura 2015).

References

Chart.js Contributors. 2026. “Chart.js Documentation.” https://www.chartjs.org/docs/latest/.

Django Software Foundation. 2026. “Django Documentation.” https://docs.djangoproject.com/.

Encode OSS Ltd. 2026. “Django REST Framework.” https://www.django-rest-framework.org/.

International Weightlifting Federation. 2026a. “International Weightlifting Federation: Competition Resources.” https://iwf.sport/.

———. 2026b. “Sinclair Coefficient Resources.” https://iwf.sport/weightlifting_/sinclair-coefficient/.

Jazzband. 2026. “Django REST Framework SimpleJWT.” https://django-rest-framework-simplejwt.readthedocs.io/.

Jones, Michael, John Bradley, and Nat Sakimura. 2015. “JSON Web Token (JWT).” RFC 7519. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519.

Meta Open Source. 2026. “React.” https://react.dev/.

PostgreSQL Global Development Group. 2026. “PostgreSQL Documentation.” https://www.postgresql.org/docs/.

SQLite Consortium. 2026. “SQLite Documentation.” https://www.sqlite.org/docs.html.

Vite Contributors. 2026. “Vite.” https://vite.dev/.

WESTMINSTER UNIVERSITY